

# One Gateway, Not Two

With the stack live on clubuntu, this laptop still ran its own local copy of the same three MCP servers. Deciding what to do about that meant correcting a wrong first assumption about how these servers were actually built.

---

decision: single cloud gateway, per-call targeting 2026-08-16

## 00 The question

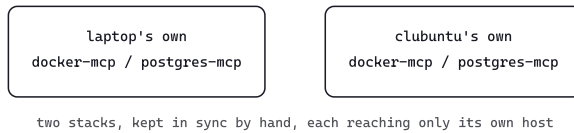
The stack now runs permanently on clubuntu. This laptop is where the source tree lives and where Claude Code is actually used day to day — and it still had its own local `docker compose up` stack running the same three MCP servers: `coder-commands-mcp`, `docker-mcp`, `postgres-mcp`. The question: keep using the local stack, or connect this laptop's Claude Code to the cloud instance instead?

## 01 The wrong frame, and the correction

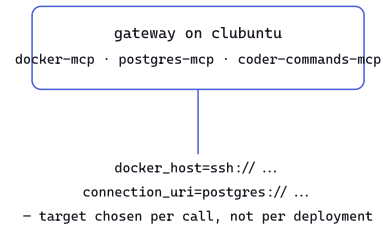
The first framing treated this as "local servers vs. cloud servers" — as if `docker-mcp` and `postgres-mcp` were tied to whichever host they happen to run on, so switching meant picking one location and losing the other. That framing was wrong, and had already been ruled out by earlier work on this project: both servers were deliberately built to take a per-call target override — `docker_host` for `docker-mcp` (an `ssh://` URL), `connection_uri` for `postgres-mcp` — specifically so one running instance could reach *any* Docker daemon or Postgres database, not just the one it happens to be deployed next to.

Once corrected, the real question wasn't "local or cloud" — it was "how many gateway instances should exist at all," and the answer became straightforward: one. A single, centrally-hosted gateway, with per-call parameters doing the work of reaching different hosts, rather than running a duplicate stack on every machine that needs the tools.

REJECTED – one stack per machine



ADOPTED – one gateway, per-call target



every machine's Claude Code talks to the same one gateway;  
reaching a different host is a call argument, never a second install

This treats all three MCP servers identically — `coder-commands-mcp` was already single-instance by nature (it only ever serves its own Postgres data); the correction was recognizing `docker-mcp` and `postgres-mcp` were built to work the same way.

## 02 Two gaps, deliberately deferred

Cross-host targeting only actually works once the target is reachable. Two real gaps were flagged, not fixed yet:

- ▶ `docker-mcp`'s own container has no SSH client or private key installed, so a `docker_host=ssh:// ...` override has nothing to authenticate outbound with.
- ▶ This laptop's local Postgres has no port published to the network, so a `connection_uri` override from the cloud `postgres-mcp` has nothing to connect to.

Both are cross-host *reachability* problems — separate from where the gateway itself runs. Agreed sequencing: switch this laptop's registrations to the cloud gateway first, since that's the immediate need, and take on the SSH/network work afterward once the switch itself is proven.

## 03 Scope: user, not project

Claude Code supports three MCP configuration scopes: `local` (the default — private, tied to one project directory), `project` (`.mcp.json`, committed and shared with anyone who clones the repo), and `user` (global, applies across every project on the machine). The existing registrations were local-scoped — only visible inside the `ManPage` project directory.

That was the wrong scope for what was actually wanted: cloud tool access from *any* project worked on this laptop, not just this one. `user` scope was the deliberate choice.

local scope (previous)

registered inside ManPage only  
other projects on this laptop:  
no access to these tools at all

user scope (adopted)

registered once, machine-wide  
every project Claude Code opens on this  
laptop reaches the same cloud gateway

Confirmed after adding: `claude mcp add ... -s user` reported "Added ... to user config" with no `[project: ...]` tag.

Scope and location are separate decisions worth naming individually — *user* vs. *project* is about which projects on this machine can see the registration; it says nothing about which host the registered server actually runs on. Both had to be decided here, not just one.

---

final-run — 1 of 3 · decision reached, execution in doc 2